BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 9

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement: Perform Image classfication using pretrained Vision Transformer (ViT model) and compare its performance with CNN Model.

Algorithm:

1. Import the required libraries, including PyTorch, Torchvision, NumPy, Matplotlib, and Scikit-learn.
2. Load the CIFAR-10 dataset containing 10 classes of images and preprocess the images by resizing and normalizing them.
3. Split the dataset into training and testing sets and create data loaders.
4. Load the pre-trained ResNet-18 CNN model and modify its final layer to classify 10 classes.
5. Load the pre-trained Vision Transformer (ViT-B/16) model and modify its classification head for 10 classes.
6. Train both models on the same training dataset for a fixed number of epochs.
7. Evaluate both models on the test dataset using accuracy, precision, recall, F1-score, and test loss.
8. Generate training loss curves and confusion matrices to visualize and compare model performance.
9. Prepare a comparison table based on classification metrics and training time.
10. Analyze the results and conclude which model performs better for the given image classification task.
11. Provide new news sentences as input to the trained model.
12. Predict the category of each new news sentence.
13. Save the fine-tuned BERT model and tokenizer.
14. Stop.

Code :

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

from torch.utils.data import DataLoader, Subset
from torchvision.models import (
    resnet18, ResNet18_Weights,
    vit_b_16, ViT_B_16_Weights
)
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)

#  Import libraries and check the available computing device.

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
print("PyTorch version:", torch.__version__)
print("Torchvision version:", torchvision.__version__)
print("Using device:", device)

if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))

#  Define preprocessing transformations and load the CIFAR-10
dataset.

mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
```

```
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])

train_data = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True,
    transform=train_transform
)

test_data = torchvision.datasets.CIFAR10(
    root="./data", train=False, download=True,
    transform=test_transform
)

classes = train_data.classes

# *Step 3: Select reproducible training and testing subsets.*
# *The same images are used to compare both models fairly.*
TRAIN_COUNT = 2000
TEST_COUNT = 500

rng = np.random.default_rng(42)
train_indices = rng.choice(len(train_data), TRAIN_COUNT,
replace=False)
test_indices = rng.choice(len(test_data), TEST_COUNT, replace=False)

train_subset = Subset(train_data, train_indices)
test_subset = Subset(test_data, test_indices)

batch_size = 8
pin = device.type == "cuda"

train_loader = DataLoader(
    train_subset, batch_size=batch_size,
    shuffle=True, num_workers=0, pin_memory=pin
)

test_loader = DataLoader(
    test_subset, batch_size=batch_size,
    shuffle=False, num_workers=0, pin_memory=pin
)
```

```
print("Classes:", classes)
print("Training images:", len(train_subset))
print("Testing images:", len(test_subset))

#  Display sample images from the CIFAR-10 dataset.*

def show_samples(dataset, indices, count=10):
    fig, axes = plt.subplots(2, 5, figsize=(13, 6))

    for ax, idx in zip(axes.flat, indices[:count]):
        image, label = dataset[int(idx)]
        image = image.permute(1, 2, 0).numpy()
        image = np.clip(
            image * np.array(std) + np.array(mean), 0, 1
        )

        ax.imshow(image)
        ax.set_title(classes[label])
        ax.axis("off")

    plt.suptitle("Sample Images from CIFAR-10")
    plt.tight_layout()
    plt.show()

show_samples(train_data, train_indices)

# Load the pretrained CNN model, ResNet-18.
# Freeze pretrained parameters and replace the final layer with 10
outputs.

cnn_model = resnet18(weights=ResNet18_Weights.DEFAULT)

for parameter in cnn_model.parameters():
    parameter.requires_grad = False

cnn_model.fc = nn.Linear(cnn_model.fc.in_features, 10)
cnn_model = cnn_model.to(device)

print("Pretrained CNN model loaded successfully.")

# Load the pretrained Vision Transformer, ViT-B/16.
# Replace its classification head to predict the 10 CIFAR-10
classes.
```

```
vit_model = vit_b_16(
    weights=ViT_B_16_Weights.IMAGENET1K_V1
)

for parameter in vit_model.parameters():
    parameter.requires_grad = False

vit_model.heads.head = nn.Linear(
    vit_model.heads.head.in_features, 10
)

vit_model = vit_model.to(device)

print("Pretrained Vision Transformer loaded successfully.")

#  Define the training function.
# Train the classification head and record loss, accuracy and
execution time.

def train_model(model, loader, epochs=2, learning_rate=1e-3):
    criterion = nn.CrossEntropyLoss()

    optimizer = optim.AdamW(
        (p for p in model.parameters() if p.requires_grad),
        lr=learning_rate
    )

    history = {"loss": [], "accuracy": []}
    start_time = time.time()

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)
            loss = criterion(outputs, labels)
```

```
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(dim=1) ==
labels).sum().item()
            total += labels.size(0)

        epoch_loss = running_loss / total
        epoch_accuracy = 100 * correct / total

        history["loss"].append(epoch_loss)
        history["accuracy"].append(epoch_accuracy)

        print(
            f"Epoch {epoch + 1}/{epochs} | "
            f"Loss: {epoch_loss:.4f} | "
            f"Training Accuracy: {epoch_accuracy:.2f}%"
        )

    training_time = time.time() - start_time

    return model, history, training_time

# Train both models on the same training subset.*

EPOCHS = 2

print("\nTraining CNN (ResNet-18)...")
cnn_model, cnn_history, cnn_time = train_model(
    cnn_model, train_loader, epochs=EPOCHS
)

print("\nTraining Vision Transformer (ViT-B/16)...")
vit_model, vit_history, vit_time = train_model(
    vit_model, train_loader, epochs=EPOCHS
)

print("\nTraining completed.")
print(f"CNN training time: {cnn_time:.2f} seconds")
print(f"ViT training time: {vit_time:.2f} seconds")

#Evaluate both models using classification metrics.*

def evaluate_model(model, loader):
```

```
    model.eval()
    all_labels = []
    all_predictions = []
    total_loss = 0.0
    total = 0

    criterion = nn.CrossEntropyLoss()

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)
            predictions = outputs.argmax(dim=1)

            total_loss += loss.item() * images.size(0)
            total += labels.size(0)

            all_labels.extend(labels.cpu().numpy())
            all_predictions.extend(predictions.cpu().numpy())

    metrics = {
        "Accuracy (%)": 100 * accuracy_score(
            all_labels, all_predictions
        ),
        "Precision (macro)": precision_score(
            all_labels, all_predictions,
            average="macro", zero_division=0
        ),
        "Recall (macro)": recall_score(
            all_labels, all_predictions,
            average="macro", zero_division=0
        ),
        "F1-score (macro)": f1_score(
            all_labels, all_predictions,
            average="macro", zero_division=0
        ),
        "Test Loss": total_loss / total
    }

    return metrics, all_labels, all_predictions

print("\nEvaluating CNN...")
```

```
cnn_metrics, y_true_cnn, y_pred_cnn = evaluate_model(
    cnn_model, test_loader
)

print("\nEvaluating Vision Transformer...")
vit_metrics, y_true_vit, y_pred_vit = evaluate_model(
    vit_model, test_loader
)

# Display the performance comparison table.*

comparison = pd.DataFrame({
    "Metric": list(cnn_metrics.keys()) + ["Training Time
(seconds)"],
    "CNN (ResNet-18)": list(cnn_metrics.values()) + [cnn_time],
    "Vision Transformer": list(vit_metrics.values()) + [vit_time]
})

print("\nPerformance Comparison")
print(comparison.round(4).to_string(index=False))

#Plot training loss curves for both models.*

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1), cnn_history["loss"],
    marker="o", label="CNN (ResNet-18)"
)

plt.plot(
    range(1, EPOCHS + 1), vit_history["loss"],
    marker="o", label="Vision Transformer"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss Comparison")
plt.xticks(range(1, EPOCHS + 1))
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#  Generate confusion matrices to examine classification errors.*
```

```
fig, axes = plt.subplots(1, 2, figsize=(17, 7))

for ax, y_true, y_pred, title in [
    (axes[0], y_true_cnn, y_pred_cnn, "CNN (ResNet-18)"),
    (axes[1], y_true_vit, y_pred_vit, "Vision Transformer")
]:
    cm = confusion_matrix(y_true, y_pred, labels=range(10))

    display = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=classes
    )

    display.plot(
        ax=ax, xticks_rotation=45, colorbar=False
    )
    ax.set_title(title)

plt.tight_layout()
plt.show()

#Print class-wise precision, recall and F1-score reports.*

print("\nCNN Classification Report")
print(classification_report(
    y_true_cnn, y_pred_cnn,
    target_names=classes, zero_division=0
))

print("\nVision Transformer Classification Report")
print(classification_report(
    y_true_vit, y_pred_vit,
    target_names=classes, zero_division=0
))

# Compare CNN and ViT predictions on one test image.*

image, actual_label = test_data[int(test_indices[0])]
batch_image = image.unsqueeze(0).to(device)

cnn_model.eval()
vit_model.eval()

with torch.no_grad():
```

```
    cnn_prediction = cnn_model(batch_image).argmax(dim=1).item()
    vit_prediction = vit_model(batch_image).argmax(dim=1).item()

original = image.permute(1, 2, 0).numpy()
original = np.clip(
    original * np.array(std) + np.array(mean), 0, 1
)

plt.figure(figsize=(5, 5))
plt.imshow(original)
plt.axis("off")
plt.title(
    f"Actual: {classes[actual_label]}\n"
    f"CNN: {classes[cnn_prediction]}\n"
    f"ViT: {classes[vit_prediction]}"
)
plt.show()

# Save the trained models and comparison results.*

torch.save(cnn_model.state_dict(), "cnn_resnet18.pth")
torch.save(vit_model.state_dict(), "vit_model.pth")
comparison.to_csv("model_comparison.csv", index=False)

print("\nSaved files:")
print("1. cnn_resnet18.pth")
print("2. vit_model.pth")
print("3. model_comparison.csv")

# *Result: Compare the measured metrics and use them to write the
conclusion.*
print("\nExperiment completed successfully.")
```

Output :

- **System Config**

```
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch version: 2.13.0+cpu
Torchvision version: 0.28.0+cpu
Device: cpu
```

- **Distinct classes**

```
Using device: cpu
DataLoaders updated successfully.
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
Training images: 10000
Testing images: 2000
```

- **Samples**



- **CNN Models training**

```
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=10, bias=True)
)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
      )
    )
```

- **Visualization:**

